Master's Theorem

Decrease function:

$$T(n) = aT(n-b) + f(n), \quad a,b > 0$$

$f(n) \rightarrow n^k \ldots$ (Gother func can also appear)

Common Mistakes:

$$T(n) = aT(n \pm b) - f(n)$$

- $T(n \pm b)$: If you add 'b' then you will never reach the base case
- $-f(n)$: The cost of composition will never substract from the cost of decomposition. It will always add.

- Be careful about this, sir may use this to trip you.

Case 01:

if $a = 1$ then

$$\text{complexity} = O(n * f(n))$$

For example,

$$T(n) = T(n-1) + n$$

So,

$$O(n * n) \Rightarrow O(n^2)$$

Case 02:

if $a > 1$ then

$$\text{complexity} = O(n^k \cdot a^{n/b})$$

For example,

$$T(n) = 2T(n-1) + 1$$

Rewrite as:

$$= \underset{a}{2}T(n-\underset{b}{1}) + 1n^{\overset{k}{0}}$$

So,

$$O(n^0 * 2^{n/1}) \Rightarrow O(2^n)$$

Case 03:

if $0 < a < 1$ then

complexity = $O(f(n))$

For example,

$T(n) = \frac{1}{2}T(n-1) + \log n$

So,

$O(\log n)$

Divide function

$T(n) = aT(n/b) + f(n)$, $a \geq 1$, $b > 1$

$f(n)$ → $n^k \log^p n$ ...

→ Gothes funcs can also appear

Common Mistakes

$T(n) = aT(n/1) + f(n)$

→ Be careful! you can't reach base case!

Case 01:

if $\log_b a > k$ then

complexity = $O(n^{\log_b a})$

For example,

$T(n) = 4T(n/2) + 1$

Rewrite as:

$= 4T(n/2) + 1n^0 \log^0 n$

$a = 4$, $b = 2$, $k = 0$, $p = 0$

Since

$\log_2 4 > 0$ we can use this case

Thus,

$O(n^{\log_2 4}) \Rightarrow O(n^2)$

Case 02:

if $\log_b a = k$ then

Case 1:

if $p > -1$ then

complexity $O(n^k \log^{p+1} n)$

For example,

$T(n) = 2T(n/2) + n$

Rewrite as:

$= 2T(n/2) + n^1 \log^0 n$

$a=2, b=2, k=1, p=0$

Since

$\log_2 2 = 1$ we can use this case

Thus,

$O(n^1 \log^{0+1} n) \Rightarrow O(n \log n)$

Case ii: $\rightarrow f(n) = \frac{n}{\log n} \Rightarrow n \log^{-1} n$

if $p=-1$ then

complexity $= O(n^k (\log(\log n)))$

case iii $\rightarrow f(n) = n/\log^2 n \Rightarrow n \log^{-2} n$

if $p < -1$ then

complexity $= O(n^k)$

Case 03:

case i:

if $-1 < p < 0$ then

complexity $= O(n^k)$

case ii

if $P \geq 0$ then

complexity $= O(f(n))$